Design Document

Hexgame: HexaHaven

January 21st, 2026

Nathan Hu (261147733),
Barry Lu (261091921),
Emily Zhang (261177806),
Helena Clifford (261104576),
Brandon Lee Felix (261234731),
Austin Wang (260349977),
James Huang (261140630)

The Little Vybihals

Contents

1 - Definitions 4
1.1 Development Cycle 4
1.1.1 Macro Development Model 4
1.1.2 Team Workflow Model 4
1.2 Engineering Procedures 5
1.3 Engineering Constraints and Assumptions 5
1.3.1 Engineering Constraints 5
1.3.2 Engineering Assumptions 6
1.4 Engineering Success and Failure Statements 6
2 - Solution Architecture [Barry & Emily 2.1] 7
2.1 Development Environment 7
2.2 Staging Environment 8
2.2.1 Purpose 8
2.2.2 Architecture Consistency 9
2.2.3 Data Isolation 9
2.2.4 Usage During Project Lifecycle 9
2.3 Deployment Environment 9
2.3.1 Client Runtime Environment 9
2.3.2 Server Runtime Environment 10
2.3.3 Installation Procedure 10
2.3.4 Configuration Strategy 10
2.3.5 Infrastructure Simplicity 11
2.4 Evolution Environment 12
2.4.1 Post-Deployment Support 12
2.4.2 Upgrade and Version Management 12
2.4.4 Retirement and Archival Plan 12
3 - Software Design 13
3.1 Storyboard and Critical Sections 13
3.1.1 Storyboard (Version 2) 13
3.1.2 Updated Storyboard Flow 14
3.2 Code Structure Identification 16
3.2.1 System-Wide Structures 16
3.2.2 Module-Based Structures 16
3.2.3 Procedure-Based Structures 17
3.3 UML Domain Model 19
3.4 Module Breakdown 19

Client 19
Server 19
4 - Database, File and API Definitions 20
4.1 Database Diagrams 20
4.2 Database Schemas 20
4.2.1 Collection: /games/{gameId} 20
4.2.2 GameConfig (Nested Map) 21
4.2.3 GameState (Nested Map) 22
4.2.4 ResourceBundle (Nested Map) 22
4.2.5 Subcollection: /games/{gameId}/players/{playerId} 22
4.2.6 Goal (Nested Map) 23
4.2.7 PlayerStats (Nested Map) 24
4.2.8 PresenceInfo (Nested Map) 24
4.2.9 Subcollection: /games/{gameId}/board/state/tiles/{tileId} 24
4.2.10 Subcollection: /games/{gameId}/board/state/structures/{structureId} 25
4.2.11 Subcollection: /users/{userId}/activeGames/{gameId} 26
4.2.12 Subcollection: /games/{gameId}/turns/{turnId} 27
4.2.13 Collection: /users/{userId} 28
4.2.14 Important Firestore Operations (Queries/Writes) 28
4.2.15 Indexing (Firestore) 29
4.3 File Definitions 29
4.3.1 Configuration Files 29
4.3.2 Data Files 30
4.4 API Definitions 30
4.4.1 Connection and Section APIs 30
4.4.2 Gameplay Action APIs 35
4.4.3 State Synchronization APIs 40
4.4.4 Error and Validation Responses 42
4.4.5 API Design Rationale 43
5 - Critical Sections 43
5.1 Risk Analysis 43
5.2 Mitigation Models 43
5.2.1 Game Session Creation 43
5.2.2 Room Join Validation 44
5.2.3 Real-time State Synchronization 45
5.2.4 Player Action Validation 46
5.2.5 Disconnect Handling 47
5.2.6 Game End Finalization 48

6 Appendix.....50
Appendix A - Contribution Table..... 50
Appendix B — References..... 50
Appendix C — Digital Source Locations..... 50
Appendix D — Index..... 51

1 - Definitions

1.1 Development Cycle

1.1.1 Macro Development Model

HexaHaven follows an iterative development with controlled spiral risk-management overlay.
The high-level system architecture (client/server separation, firestore persistence, Socket.io communication) was established early based on the specifications document.
Development then proceeds in structured feature increments:

1. Lobby system and basic board rendering
2. Turn state machine and dice rolling
3. Resource distribution and build validation
4. Goal detection and win condition
5. Save/load validation and stabilization

Waterfall characteristics are applied at milestone checkpoints. Spiral elements are used when implementing high-risk components such as:

- Turn synchronization
- Dice/resource validation
- Save/load validation
- Multiplayer state consistency

This hybrid model ensures academic milestone compliance while actively mitigating technical risk.

1.1.2 Team Workflow Model

The team uses a Standard incremental workflow with peer review and staging validation. Each increment follows this structured cycle:

<u>Planning Phase</u>

- Weekly coordination meeting
- Task allocation aligned with Gantt chart
- Definition of acceptance criteria

<u>Implementation Phase</u>

- Development occurs on feature branches
- Server logic implemented first for authoritative validation
- Client rendering and interaction implemented afterward

<u>Validation Phase</u>

- Unit testing using Jest
- Manual gameplay testing

- Staging validation before merge

Integration Phase

- Pull request review
- Merge to main branch
- Internal demonstration

This workflow supports controlled integration and consistent system stability.

1.2 Engineering Procedures

Version Control

- All source code is managed using Git.
- Development occurs on isolated feature branches.
- Pull requests are required before merging.
- Commit messages must clearly describe functional changes.

Code Review

Each pull request must be reviewed by at least one team member. Review criteria include:

- Compliance with authoritative server architecture
- Correct enforcement of turn-based gameplay
- Absence of direct client-side state mutation
- Schema consistency with firestore persistence model

Testing

Testing includes:

- Unit tests for:
    - Turn state machine transitions
    - Dice roll correctness (2–12 inclusive)
    - Resource distribution
    - Goal completion validation
- Integration testing in staging
- Save/load compatibility checks

Critical sections must pass validation before merging.

Documentation Discipline

- Any schema change requires documentation update.
- API changes must be reflected in both specification and design documents.
- Save file schema version must be incremented if structure changes.

This ensures traceability between specification and implementation.

1.3 Engineering Constraints and Assumptions

1.3.1 Engineering Constraints

Item	Constraint	Expected Impact	Mitigation Strategy
Academic Timeline	Must complete within	Limited feature scope	Prioritize core

	the winter semester		gameplay loop
No dedicated infrastructure	Runs locally without cloud scaling	Limited stress testing	Simulated staging validation
Multiplayer synchronization	Real-time consistency required	Increased logic complexity	Server-authoritative enforcement
Firestore persistence	File-based storage only	Requires careful schema validation	Schema version control
Browser-based runtime	Must run in desktop browsers	Client rendering constraints	Limit to supported browsers

1.3.2 Engineering Assumptions

Item	Assumption	Impact	Mitigation
Stable WebSocket Connection	Multiplayer relies on Socket.io	Risk of desync	Server validation on every action
Maximum 4 players	Limited concurrency	Simplifies synchronization	Hard core validation
Short session gameplay	No long-term data retention	Simplifies storage	Snapshot-based saves
No AI opponents	Only human multiplayer	Reduced complexity	Clear rule enforcement

1.4 Engineering Success and Failure Statements

Item	Success Condition	Expected Impact	Discussion
Multiplayer Stability	No state divergence across clients	Reliable gameplay	Server-authoritative design
Turn integrity	Only active player can act	Fairness	Strict phase validation
Dice & Resource Logic	Correct resource allocation	Functional economy	Defensive validation checks
Save/Load Validation	Compatible and consistent restore	Data integrity	Schema version enforcement
Milestone Delivery	All demos functional	Academic success	Scope prioritization

Item	Failure Condition	Impact	Mitigation
Desynchronization	Client/server state mismatch	Invalid gameplay	Centralized validation
Phase violations	Actions outside correct turn phase	Broken game logic	Phase guard checks
Corrupted save	Invalid firestore load	Data loss	Validation + error handling
Missed deadlines	Incomplete milestone delivery	Academic penalty	Prioritized backlog

2 - Solution Architecture [Barry & Emily 2.1]

2.1 Development Environment

Item	Name	Version	Description/Environment	Quantity
Operating System	Windows / macOS / Linux	Latest stable	Developer workstation OS	1 per developer
Programming Language	TypeScript	Latest stable	Client and server implementation	N/A
Runtime	Node.js	18+ (LTS)	Executes server-side code	1
Web Framework	Express.js	Latest stable	HTTP server and REST APIs	1
Real-Time Networking	Socket.io	Latest stable	WebSocket communication	1
Frontend Rendering	HTML Canvas	Browser-provided	Hex-based board rendering	1
Styling framework	Tailwind CSS	Latest stable	UI styling	1
Version Control	Git	Latest Stable	Source code management	1
Repository Hosting	GitHub	N/A	Centralized code repository	1

IDE	VSCode	Latest Stable	Development editor	1
Database	Firebase Firestore	15+	Persistent storage for game data	1
Testing Framework	Jest	Latest Stable	Unit and integration testing	1

Installation Procedure
To set up the HexaHaven development environment, a developer performs the following steps: 1. Install Node.js (version 18 or later). 2. Install Git. 3. Clone the HexaHaven GitHub repository. 4. Install project dependencies using npm install. 5. Install and configure a local database instance. 6. Initialize the database using the provided schema or migration scripts. 7. Configure environment variables (e.g., database connection, server port). 8. Start the development server using npm run dev. 9. Open a web browser and access the local development URL.

Expected Workflow
The expected workflow for developers is as follows: 1. Pull the latest changes from the main repository. 2. Create or switch to a feature branch. 3. Implement changes in either client or server code. 4. Run the server locally and test gameplay through the browser. 5. Execute unit and integration tests. 6. Commit changes with descriptive messages. 7. Push the branch to GitHub. 8. Open a pull request for review and integration. Development follows an iterative and incremental approach, allowing features to be developed, tested, and integrated continuously throughout the project lifecycle.

2.2 Staging Environment

2.2.1 Purpose

The staging environment is used to validate integrated system behavior before formal demonstrations and milestone evaluations. It mirrors the production architecture while isolating test data from live or demonstration sessions.

This environment enables validation of:

- Firestore read/write operations
- Real-time synchronization across multiple clients
- Turn state enforcement
- Spectator read-only behavior
- Disconnect handling
- Game state persistence and restoration

2.2.2 Architecture Consistency

TThe staging environment mirrors the deployment architecture described in Section 2.3:

- A Node.js (v18+) authoritative server executes game logic.
- Express handles REST endpoints for session initialization.
- Socket.io manages real-time communication.
- Firestore (via Firebase Emulator or separate staging project) stores session state.
- Clients connect through modern web browsers.

By maintaining architectural parity, staging accurately reflects production behavior while allowing safe validation of persistence logic.

2.2.3 Data Isolation

Staging uses either:

- Firebase Emulator Suite (local Firestore emulator), or
- A dedicated Firestore staging project.

This ensures:

- Test sessions do not pollute production data.
- Schema evolution can be tested safely.
- Security rules can be validated before deployment.
- Session cleanup policies can be tested.

All staging sessions are either soft-deleted or automatically expired after testing.

2.2.4 Usage During Project Lifecycle

The staging environment is used for:

- Integration testing after feature merges
- Validation of Firestore schema updates
- Testing composite indexes
- Verifying security rule behavior
- Interim demonstrations
- Pre-submission validation

No feature is considered stable unless it passes staging validation with Firestore persistence enabled.

2.3 Deployment Environment

2.3.1 Client Runtime Environment

The client runtime consists of:

- Modern desktop web browser (Chrome, Firefox, Safari)
- HTML/CSS UI
- HTML Canvas rendering layer
- TypeScript-compiled JavaScript
- Firebase client SDK for Firestore access
- Socket.io client for real-time communication

Clients never modify game state directly. All write operations are validated server-side.

2.3.2 Server Runtime Environment

The server runtime environment consists of:

- Node.js (v18+ LTS)
- Express.js
- Socket.io
- Firebase Admin SDK
- Firestore (cloud-hosted NoSQL database)

The server is authoritative and responsible for:

- Turn enforcement
- Action validation
- Dice roll computation
- Resource distribution
- Goal completion detection
- Firestore transaction handling
- Synchronization broadcasting

Persistent storage is handled exclusively through Firestore collections and subcollections defined in Section 4.

No file-based persistence is used in production.

2.3.3 Installation Procedure

To deploy HexaHaven:

- Install Node.js (v18+).
- Clone the GitHub repository.
- Install dependencies (npm install).
- Configure Firebase project credentials.
- Set environment variables (Firebase project ID, service account key).
- Deploy Firestore rules and indexes.
- Start the server (npm run start).

- Access via browser.

2.3.4 Configuration Strategy

Deployment configuration is managed via:

- Environment variables (.env)
- Firebase project configuration
- Firestore security rules
- Composite index definitions

Server startup validates:

- Firebase Admin SDK initialization
- Firestore connectivity
- Required indexes existence

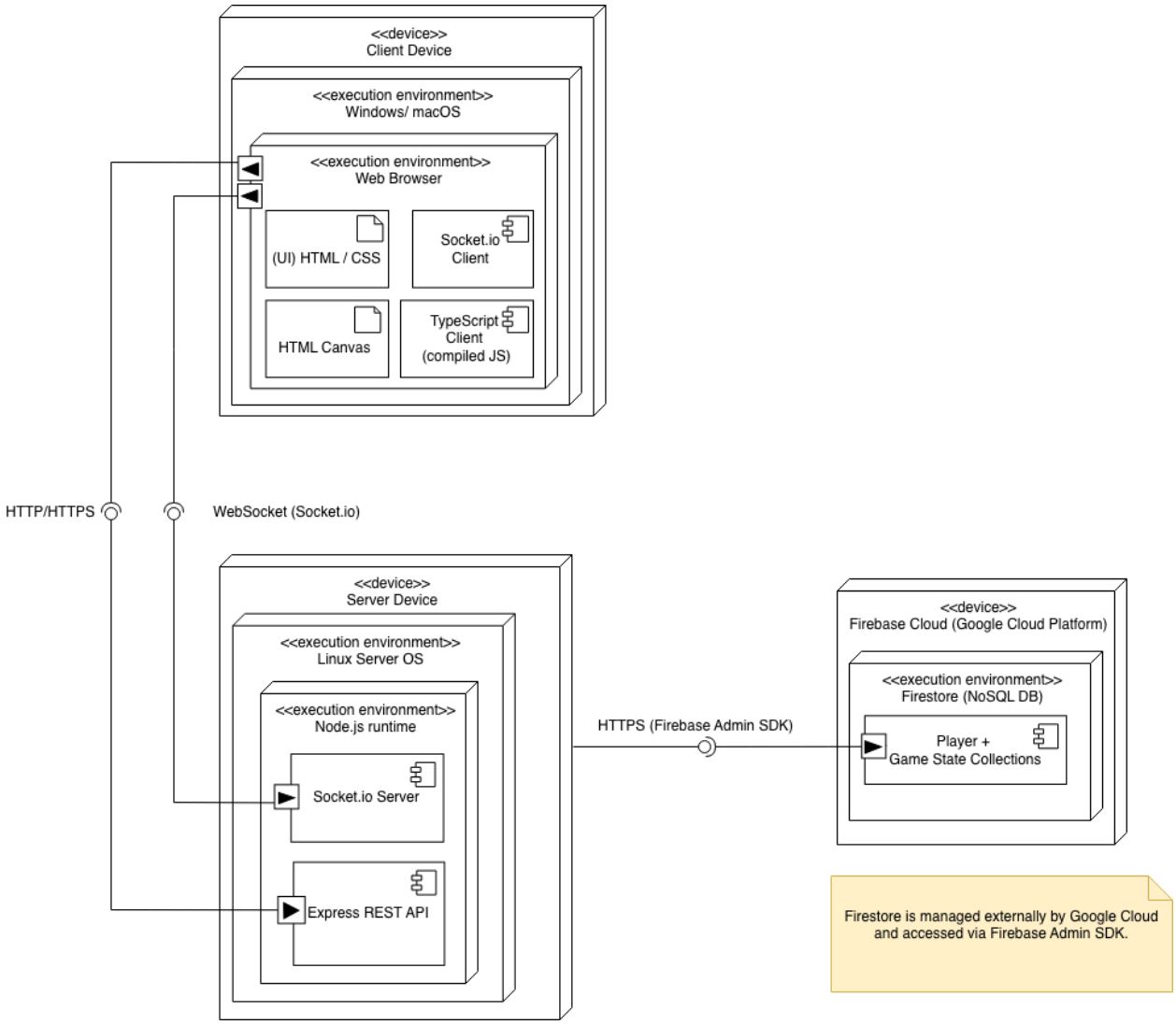
If validation fails, the server terminates to prevent inconsistent runtime behavior.

2.3.5 Infrastructure Simplicity

HexaHaven relies on:

- Cloud-hosted Firestore
- WebSocket-based real-time synchronization
- Server-authoritative validation model

All session persistence is handled via structured Firestore collections defined in Section 4.



2.4 Evolution Environment

2.4.1 Post-Deployment Support

Issues are first reproduced in the development environment and validated in staging using the Firebase Emulator or staging Firestore project before being applied to production.

Bug categories include:

- Firestore write conflicts
- State synchronization errors
- Turn-phase violations
- Security rule rejections

Because the server is authoritative, all rule corrections occur server-side to maintain consistency.

2.4.2 Upgrade and Version Management

HexaHaven uses structured Firestore schemas.

Schema evolution is managed through:

- Versioned document structures
- Backward-compatible field additions
- Controlled migration scripts (if necessary)
- Firestore rules updates

New features are implemented in development, validated in staging, then deployed via Git version tagging.

2.4.3 Feature Evolution Strategy

Future enhancements (e.g., new goals, UI improvements, rule variations) follow the modular architecture:

- Client updates affect rendering and interaction only.
- Server updates affect rule validation and state transitions.
- Persistence structure changes require schema updates and rule validation.

All changes must preserve:

- Turn integrity
- Real-time synchronization
- Firestore consistency
- Security rule compliance

2.4.4 Retirement and Archival Plan

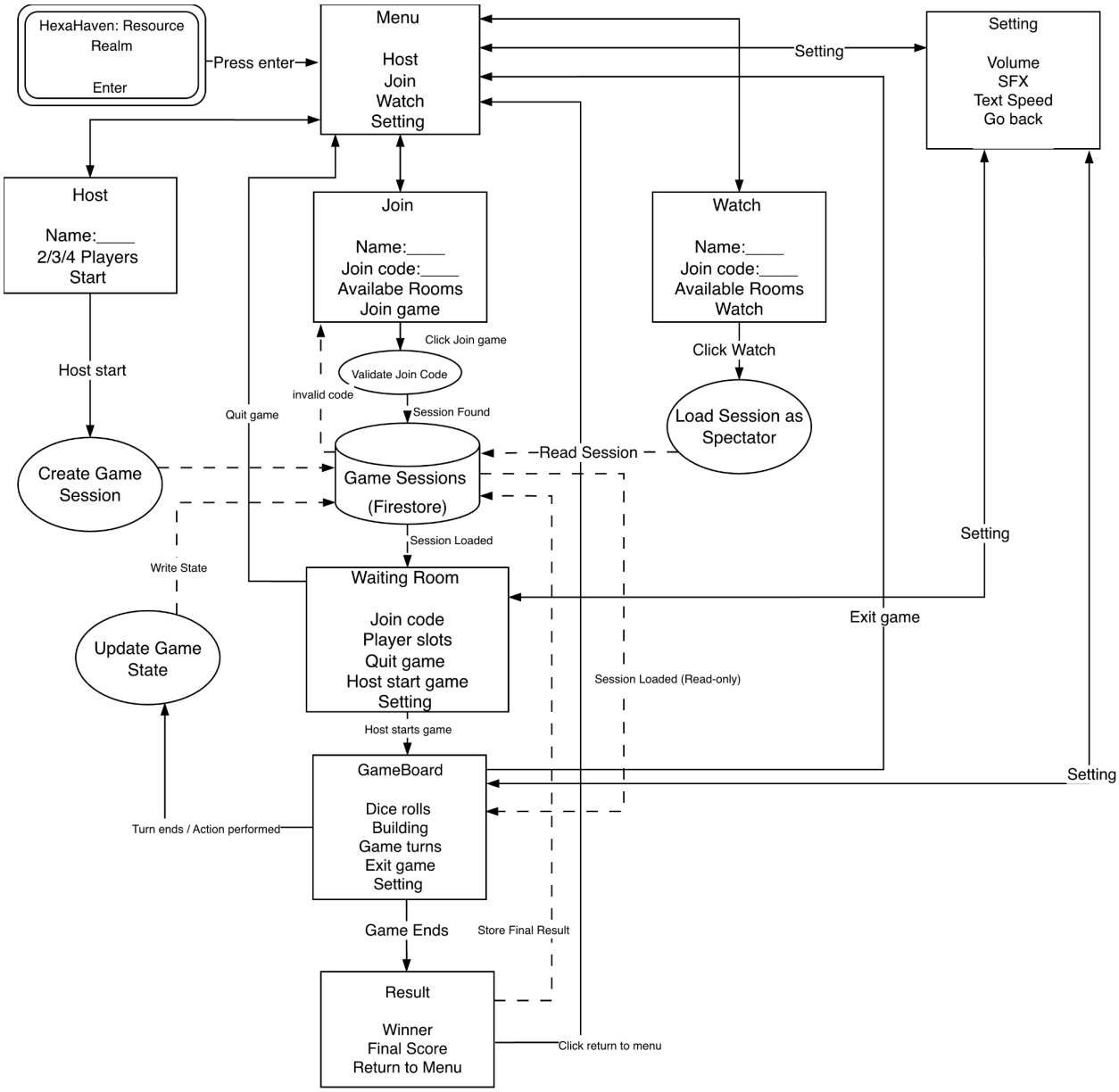
If the system is retired:

- Firestore collections may be exported using Firebase export tools.
- Source code remains version-controlled.
- No relational database migration is required.
- Session documents may be archived as JSON exports for recordkeeping.

3 - Software Design

3.1 Storyboard and Critical Sections

3.1.1 Storyboard (Version 2)



The storyboard represents the macro-level structure of HexaHaven: Resource Realm from both the user's and system's perspective. It illustrates the primary screens, transitions, and session lifecycle interactions experienced within the multiplayer environment.

While the storyboard presents user-facing navigation flow, it also explicitly models backend interactions involving Firestore persistence, server-authoritative validation, real-time synchronization, spectator read-only access, and disconnect handling.

The storyboard is organized into the following major user-facing modules:

- Entry Screen
- Main Menu
- Host Game
- Join Game
- Watch Game (Read-only Spectator Mode)
- Waiting Room (Lobby)
- GameBoard (Core Gameplay)
- Result Screen
- Settings Menu

In addition to UI transitions, the storyboard highlights:

- Session creation and join validation through Firestore
- Real-time state synchronization via Socket.io
- Server-side action validation
- Persistent session state updates
- Final result storage and integrity enforcement

This ensures that the storyboard reflects both the user experience and the underlying distributed system architecture.

3.1.2 Updated Storyboard Flow

The application begins at the Entry Screen, where the user presses enter to proceed to the Main Menu. From the Main Menu, the player may choose to host a game, join an existing session, watch an active session as a spectator, or adjust settings.

If the user selects Host, the system generates a unique join code and creates a new session document in Firestore. Session initialization is handled server-side to prevent duplicate sessions and ensure data integrity. Once successfully created, the session state is loaded into the Waiting Room.

If the user selects Join, the provided join code is validated against Firestore. The system verifies that the session exists and has available player slots before admitting the user. If validation fails, the user is redirected back to the Join screen with an error state. If validation succeeds, the session is loaded into the Waiting Room.

If the user selects Watch, the system performs a read-only retrieval of an active session from Firestore. Spectators are attached to the GameBoard in read-only mode and cannot modify game state. This ensures that spectator participation does not interfere with gameplay integrity.

Within the Waiting Room, player readiness and session membership are synchronized. When the host initiates the match, all connected players transition to the GameBoard.

During gameplay, all player actions (e.g., dice rolls, building, turn progression) are validated server-side. Valid actions trigger state updates that are written to Firestore and broadcast to connected clients via Socket.io to maintain consistent game state across all participants.

If a player disconnects during a session, the system preserves game integrity by maintaining authoritative state on the server and synchronizing remaining clients.

When the game concludes, final scores are computed server-side and written to Firestore to prevent duplicate or conflicting result submissions. The system then transitions to the Result Screen, where players may return to the Main Menu.

Developer's Critical Section Identification Table

Items	Title	Description	Location	Owner
1	Game Session Creation	Generate unique join code and create Firestore session safely	Host → Server	Developer
2	Room Join Validation	Ensure session exists and is not full before adding player	Join → Waiting Room	Developer
3	Real-time State Synchronization	Ensuring all players receive consistent game state updates via Socket.io	GameBoard	Developer
4	Player Action Validation	Validate all client actions server-side to prevent illegal moves	Server Socket Layer	Developer
5	Disconnect Handling	Maintain game integrity if a player disconnects mid-session	Waiting Room / GameBoard	Developer
6	Game End Finalization	Preventing duplicate result computation and multiple result writes	GameBoard → Result	Developer

3.2 Code Structure Identification

This section identifies the core technologies, architectural patterns, and data structures required to implement the multiplayer architecture of HexaHaven. The structures are organized into system-wide, module-based, and procedure-based categories.

3.2.1 System-Wide Structures

System-wide structures represent global architectural decisions, technologies, and design patterns that apply across the entire system.

Item	Title	Type	Purpose	Location
1	TypeScript / JavaScript	Technology	Implementation language for client and server	Client & Server
2	Socket.io	Technology	Enable real-time bidirectional multiplayer communication	Client & Server
3	Express	Technology	Provide REST endpoints for session management	Server
4	Firestore	Technology	Persistent NoSQL storage for session and player data	Cloud
5	Client–Server Pattern	Design Pattern	Separate client interface from authoritative server logic	Entire System
6	MVC Pattern	Design Pattern	Separate UI (View), input handling (Controller), and game state (Model)	Client
7	Observer Pattern	Design Pattern	Allow server to broadcast state updates to subscribed clients	Socket.io Layer
8	Singleton Pattern	Design Pattern	Ensure one authoritative GameServer instance manages sessions	Server

3.2.2 Module-Based Structures

Module-based structures describe the major system components and the data structures they use.

Client-side Modules

Module	Responsibility	Key Data Structures
UI Renderer	Render screens and gameplay	Tile[][] or rendered board matrix
Input Handler	Capture user interactions	Event listener registry
Networking Client	Send/receive socket events	Socket instance
Local State Mirror	Maintain temporary client state	GameState objec

Server-side Modules

Module	Responsibility	Key Data Structures
Socket Gateway	Manage real-time communication	Map<PlayerId, Socket>

Room Manager	Store and manage active game sessions	Map<String, Room>
Room	Represent a single multiplayer session	List<Player>
Game Engine	Apply rules and validate actions	GameState object
Turn Manager	Track active player and turn order	Queue<Player> or currentTurnIndex
Persistence Service	Interface with Firestore for session storage	FirestoreRepository instance

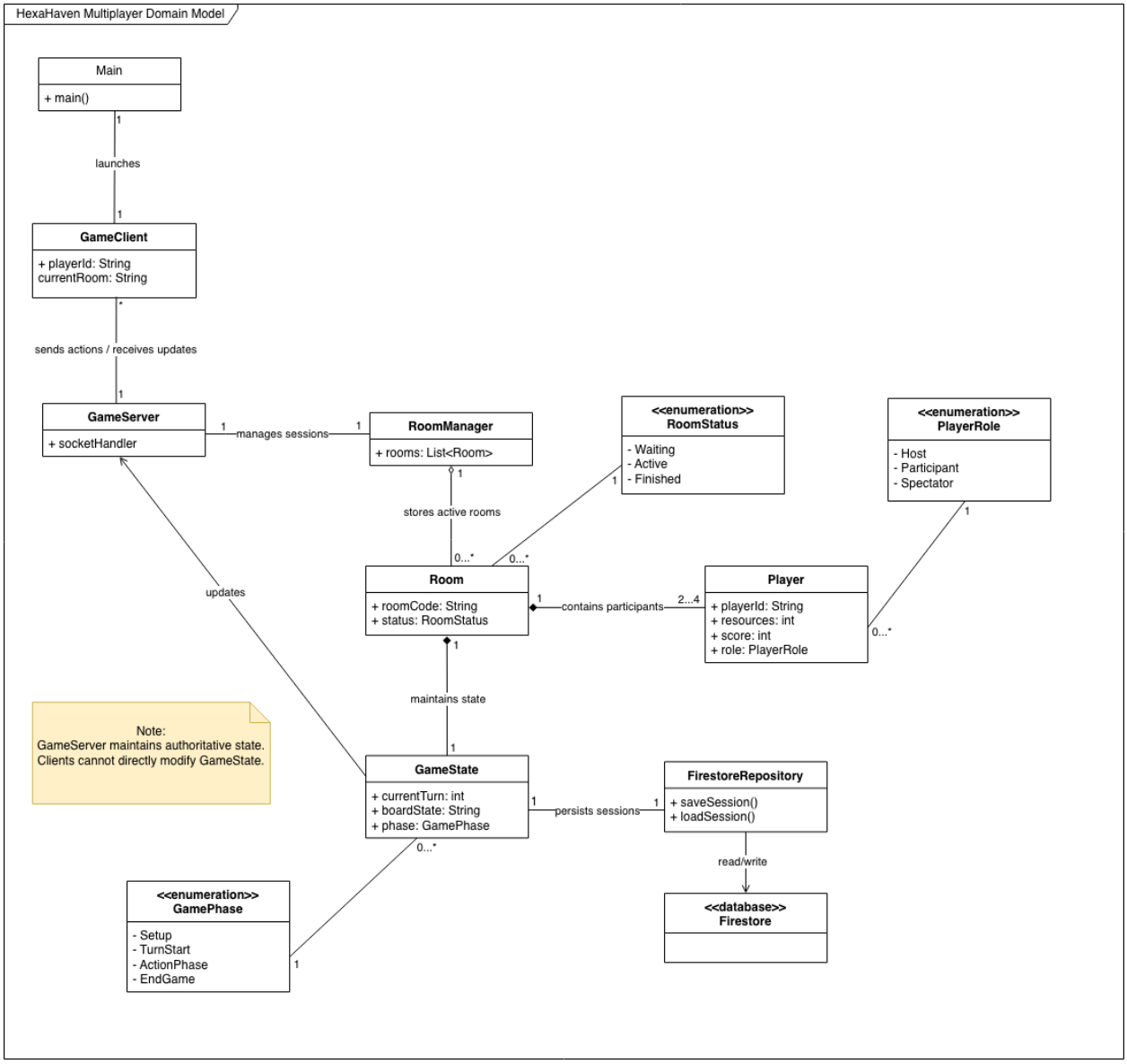
3.2.3 Procedure-Based Structures

Procedure-based structures describe processes related to development, deployment, and maintenance.

Item	Phase	Title	Purpose	Technology / Procedure	Location
1	Development	Local Server Setup	Enable local testing of multiplayer logic	Node.js dev server	Developer Environment
2	Development	Firestore Emulator Testing	Test database writes without affecting production	Firebase Emulator	Local Environment
3	Development	Game Logic Unit Testing	Ensure rules engine works independently of sockets	Unit tests	Server
4	Deployment	Environment Configuration	Configure production environment variables securely	.env configuration	Server
5	Deployment	Firestore Security Rules	Enforce production read/write restrictions	Firestore Rules	Cloud
6	Deployment	HTTPS Configuration	Secure client-server communication	SSL / HTTPS setup	Server
7	Maintenance	Session Expiration Policy	Prevent old sessions from accumulating	Firestore cleanup logic	Server

8	Maintenance	Game Rule Updates	Allow modification of gameplay logic	Versioned game engine	Server

3.3 UML Domain Model



3.4 Module Breakdown

The system is divided into client-side and server-side components.

Client

- Rendering Layer
- Input Layer

- Networking Layer

Server

- Real-Time Communication Layer
- Game Logic Engine
- Session Manager
- Persistence Layer

4 - Database, File and API Definitions

4.1 Database Diagrams

Firestore Database Structure

```
/games/{gameId}                                  [Document]
├── /players/{playerId}                          [Subcollection → Document]
├── /board                                       [Subcollection]
│   └── /state                                   [Document (singleton)]
│       ├── /tiles/{tileId}                      [Subcollection → Document]
│       └── /structures/{structureId}            [Subcollection → Document]
└── /turns/{turnId}                              [Subcollection → Document]
    (turnNumber stored as Number field in each turn document)

/users/{userId}                                  [Document]
└── /activeGames/{gameId}                        [Subcollection → Document]
```

Relationships:

- Game → Players: 1 : 2–4 (stored in /games/{gameId}/players/{playerId})
- Game → Tiles/Structures: 1 : N (stored under /games/{gameId}/board/state/tiles and /structures)
- Player → Structures: 1 : 0..* (structures reference owner via ownerPlayerId)
- Game → Turns: 1 : 0..* (stored in /games/{gameId}/turns/{turnId}, each turn references playerId and turnNumber)
- User → ActiveGames: 1 : 0..* (stored in /users/{userId}/activeGames/{gameId} for quick lookup of a user's active games)

4.2 Database Schemas

Section 4.2 specifies our Firestore database schemas, detailing each collection/document's fields, data types, required constraints, and indexing considerations to ensure valid and efficient game-state storage and retrieval.

4.2.1 Collection: /games/{gameId}

Purpose: Top-level document containing configuration and current game state.

Field	Type	Required	Constraints / Legal Values	Notes
roomCode	String	Yes	Alphanumeric, 6-10 chars, unique	Human-readable join code [colour]-[4 digit hash] Ex. “GREEN-2311]
status	String	Yes	“waiting” | “in_progress” | “finished”	Game State
config	Map	Yes	See GameConfig schema	Immutable after start
state	Map	Yes	See GameState Schema	Updated each turn
board	Map	Yes	See BoardSetup Schema	Static after creation
playerOrder	Array	Yes	Array of playerIds	Turn sequence
playerStats	Map	Yes	Map of playerId -> stats See PlayerStats Schema	
winner	String	No	playerId or null	Set when game ends
createdAt	Timestamp	Yes	Firestore Timestamp	
updatedAt	Timestamp	Yes	Firestore Timestamp	
createdBy	String	Yes	userId of host	
isDeleted	Boolean	Yes		Deletion flag to allow for recovery after “deletion”

4.2.2 GameConfig (Nested Map)

Field	Type	Required	Constraints / Legal Values	Notes
playerCount	Number	Yes	1-4	
goalCount	Number	Yes	= 3	
winRule	String	Yes	“ALL_GOALS_COMPLETE” | “ANY_X_GOALS_COMPLETE” | “FIRST_TO_X_POINTS”	
mapSeed	Number	Yes	1-32 digit integer	Used for map generation

mapSize	String	Yes	“small” | “medium” | “large”	
timerEnabled	Boolean	Yes		
turnTimeSec	Number	No	> 0, required if timerEnabled	
allowReroll	Boolean	Yes		
startingResources	Map	Yes	See ResourceBundle	The quantity for each resource that the players start with

4.2.3 GameState (Nested Map)

Field	Type	Required	Constraints / Legal Values	Notes
currentTurn	Number	Yes	≥ 0	
currentPlayerId	String	Yes	Must be in playerOrder	
currentPlayerIndex	Number	Yes	0 to playerCount-1	
phase	String	Yes	"ROLL" | "COLLECT" | "ACTION" | "END"	
turnStartedAt	Timestamp	No	Firestore Timestamp	
turnEndsAt	Timestamp	No	Firestore Timestamp	
lastDiceRoll	Map	No	{d1Val, d2Val, sum, rolledAt}	

4.2.4 ResourceBundle (Nested Map)

Field	Type	Required	Constraints / Legal Values	Notes
CRYSTAL	Number	Yes	≥ 0	
STONE	Number	Yes	≥ 0	
BLOOM	Number	Yes	≥ 0	
EMBER	Number	Yes	≥ 0	
GOLD	Number	Yes	≥ 0	

4.2.5 Subcollection: /games/{gameId}/players/{playerId}

Purpose: Individual player state within a game

Field	Type	Required	Constraints / Legal Values	Notes
userId	String	Yes	Firebase Auth UID	Links to user profile
displayName	String	Yes	1-16 chars, alphanumeric	
Colour	String	Yes	Hex color code	For UI rendering
isHost	Boolean	Yes		First player is host
resources	ResourceBundle	Yes	All ≥ 0	Current resources count
goals	Map	Yes	See Goal Schema	
stats	Map	Yes	See PlayerStats Schema	
presence	Map	Yes		Connection Status
joinedAt	Timestamp	Yes	Firestore Timestamp	
updatedAt	Timestamp	Yes	Firestore Timestamp	

4.2.6 Goal (Nested Map)

Field	Type	Required	Constraints / Legal Values	Notes
goalId	String	Yes		
type	String	Yes	"UPGRADE_SETTLEMENT" | "ROAD_PATH" | "COLLECT_RESOURCE"	
params	Map	Yes	Type-specific, see below	
completed	Boolean	Yes		
completedAtTurn	Number	Yes	≥ 0	

description	String	Yes		
progress	Number	Yes	0-100 percentage	
targetValue	Number	Yes	Goal Threshold	

Goal Params by Type:

- **UPGRADE_SETTLEMENT:** { targetLevel: number }
- **ROAD_PATH:** { minLength: number }
- **COLLECT_RESOURCE:** { resource: ResourceType, amount: number }

4.2.7 PlayerStats (Nested Map)

Field	Type	Required	Constraints / Legal Values	Notes
publicVP	Number	Yes	≥ 0	Victory Points, given by objectives
settlementsBuilt	Number	Yes	≥ 0	
roadsBuilt	Number	Yes	≥ 0	
totalResourcesCollected	Number	Yes	≥ 0	
totalResourcesSpent	Number	Yes	≥ 0	
longestRoadLength	Number	Yes	≥ 0	
turnsPlayed	Number	Yes	≥ 0	

4.2.8 PresenceInfo (Nested Map)

Field	Type	Required	Constraints / Legal Values	Notes
isConnected	Boolean	Yes		
lastSeenAt	Timestamp	Yes		
connectionId	String	Yes		

4.2.9 Subcollection: /games/{gameId}/board/state/tiles/{tileId}

Purpose: Static board tiles (set once at game creation)

Field	Type	Required	Constraints / Legal Values	Notes

coord	Map	Yes	{q: number, r: number}	Axial coordinates
resourceType	String	Yes	“CRYSTAL” | “STONE” | “BLOOM” | “EMBER” | “GOLD” | “DESERT”	
numberToken	Number	Yes	2-12 (null for DESERT)	Number to be rolled for resources to be given
adjacentTiles	Array	Yes	array of tileIds	adjacency list
vertices	Array	Yes	Array of 7 vertex ODs	Corner identifiers
edges	Array	Yes	Array of 6 edge IDs	Side identifiers
createdAt	Timestamp	Yes	Firestore Timestamp	

4.2.10 Subcollection: /games/{gameId}/board/state/structures/{structureId}

Purpose: Dynamic structures (roads, settlements) built during gameplay

Field	Type	Required	Constraints / Legal Values	Notes
ownerPlayerId	String	Yes	Must reference valid player	
ownerName	String	Yes		
ownerColor	String	Yes		
type	String	Yes	ROAD | SETTLEMENT | GARDEN | …	
level	Number	Yes	≥ 1	For settlement upgrades
locationType	String	Yes	EDGE | VERTEX	Determines location schema
vertex	Map	Yes	See VertexLocation schema	
edge	Map	Yes	See EdgeLocation schema	
adjacentStructures	Array	Yes	Array of structure references	For validation

adjacentTiles	Array	Yes	Array of tile references	For resource collection
builtAtTurn	Number	Yes	≥ 0	
builtAt	Timestamp	Yes	Firestore Timestamp	
cost	Map	Yes	ResourceBundle	Cost to construct
roadPath	String	Yes	See RoadPath schema	Only for roads

Location Types (stored in location Map field):

- If type ∈ {SETTLEMENT, CITY} → location follows VertexLocation JSON
- If type = ROAD → location follows EdgeLocation JSON

RoadPath (optional Map field):

- Only for type = ROAD → roadPath follows RoadPath JSON
- Otherwise roadPath is omitted / null

VertexLocation JSON

```
{
 id: string,            // Unique vertex identifier
 hex: {q: number, r: number},
 corner: number,        // 0-5
 adjacentHexes: [       // All hexes touching this vertex
  {q: number, r: number}
 ]
}
```

EdgeLocation JSON

```
{
 id: string,            // Unique edge identifier
 hex: {q: number, r: number},
 dir: number,           // 0-5
 adjacentHexes: [       // Both hexes touching this edge
  {q: number, r: number}
 ]
}
```

RoadPath JSON

```
{
 connectedRoads: [string],  // Array of connected structure IDs
 pathLength: number         // For longest road goal
}
```

4.2.11 Subcollection: /users/{userId}/activeGames/{gameId}

Purpose: quick lookup of a user's active games

Field	Type	Required	Constraints / Legal Values	Notes
gameId	String	Yes		
roomCode	String	Yes		
status	String	Yes	waiting|in_progress|finished	
lastAccessedAt	Timestamp	Yes		

4.2.12 Subcollection: /games/{gameId}/turns/{turnId}

Purpose: Action log for each turn

Field	Type	Required	Constraints / Legal Values	Notes
playerId	String	Yes		Player who took this turn
playerName	String	Yes		
status	String	Yes	"in_progress" | "completed"	
diceRoll	Map	Yes	{d1, d2, sum, rolledAt}	
actions	Array	Yes		Array of Action objects
stateBefore	Map	No		Snapshot for debugging
stateAfter	Map	No		Snapshot for debugging
startedAt	Timestamp	Yes	Firestore Timestamp	
endedAt	Timestamp	No	Firestore Timestamp	
duration	Number	Yes		Seconds elapsed.

turnNumber	Number	Yes	Integer ≥ 1	Sequential turn index used for ordering

Action (Array Element JSON)

```
{
 actionId: string,
 type: "DICE_ROLL" | "COLLECT_RESOURCES" | "BUILD_ROAD" |
     "BUILD_SETTLEMENT" | "UPGRADE_SETTLEMENT" | "GOAL_COMPLETED",
 timestamp: Timestamp,
 // Type-specific data:
 result?: object,         // For dice roll
 resources?: ResourceBundle, // For resource collection
 structureId?: string,     // For builds
 location?: object,        // For builds
 cost?: ResourceBundle,    // For builds
 goalId?: string,          // For goal completion
 goalType?: string         // For goal completion
}
```

4.2.13 Collection: /users/{userId}

Purpose: Individual players

Field	Type	Required	Constraints / Legal Values	Notes
userId	String	Yes	Firebase Auth UID	
displayName	String	Yes	1-16 chars, alphanumeric	
stats	Map	Yes	See PlayerStats Schema	
presence	Map	Yes		Connection Status
joinedAt	Timestamp	Yes	Firestore Timestamp	
updatedAt	Timestamp	Yes	Firestore Timestamp	
isDeleted	Boolean	Yes		Flag for soft deletion

4.2.14 Important Firestore Operations (Queries/Writes)

- **Create game:** Create `/games/{gameId}` with initial config and state; initialize `/games/{gameId}/board/state` and seed `tiles/structures` subcollections if applicable.

- **Join game (by roomCode):** Query games where `roomCode == <code>`; create `/games/{gameId}/players/{playerId}`; add `/users/{userId}/activeGames/{gameId}`.
- **Load game session (real-time):** `Subscribe/listen` to `/games/{gameId}` and player docs for live updates.
- **Load board state:** Read `/games/{gameId}/board/state` and its subcollections `/tiles` and `/structures`.
- **Place structure/road:** Validate placement; update `/board/state/structures/{structureId}`; update player resources in `/players/{playerId}`; append an entry in current `/turns/{turnId} actions[]`.
- **Roll dice + distribute resources:** Write `diceRoll` into the current turn doc; update affected players' resource bundles.
- **End turn:** Mark current turn status = completed and set `endedAt`; update `/games/{gameId}.state` to advance `currentPlayerId` and increment `currentTurn`; create the next `/turns/{turnId}` with `turnNumber`.
- **Turn history:** Query `/games/{gameId}/turns` ordered `by turnNumber` ascending to reconstruct past actions.
- **User dashboard:** Query `/users/{userId}/activeGames` ordered by `lastAccessedAt` descending.

4.2.15 Indexing (Firestore)

- **Games lookup by roomCode:** `games` collection index on `roomCode` (for where roomCode == <code>).
- **Turn history ordering:** `turns` subcollection index on `turnNumber` (for `orderBy("turnNumber")`).
- **Structures by owner:** `structures` subcollection index on ownerPlayerId (for where `ownerPlayerId == <playerId>`).
- **User active games ordering:** `activeGames` subcollection index on `lastAccessedAt` (for `orderBy("lastAccessedAt", "desc")`).

4.3 File Definitions

HexaHaven is Firestore-centric, so configuration is managed through Firebase project configuration, database policy files (rules/indexes), and environment-specific settings rather than user-editable INI/XML configs.

4.3.1 Configuration Files

- **firestore.rules**: Firestore security rules defining read/write permissions and integrity constraints for game sessions (games, players, board state, turns).
- **firestore.indexes.json**: Firestore composite index definitions required by planned query patterns (e.g., roomCode lookup, ordering turns by turnNumber, filtering by ownerPlayerId).

- **firebase.json**: Firebase deployment configuration specifying which rules/index definitions are applied.
- **.firebaserc**: Firebase project mapping for development/production environments.
- **firebaseConfig.ts**: Firebase client SDK initialization values used by the frontend to connect to the correct project.

4.3.2 Data Files

HexaHaven stores all authoritative gameplay and user data in Firestore; therefore, the system does not rely on external CSV/JSON/XML files for persistent game-state storage. The only non-database "data files" are supporting assets and optional client-side cached data.

- **Static client assets (images/icons/audio/fonts)**: Stored in the client assets directory and bundled with the frontend for UI rendering.

4.4 API Definitions

All APIs are event-driven and transmitted over persistent WebSocket connections using Socket.io. The client does not modify game state directly; instead, it issues validated requests that the server processes and applies to the authoritative game state.

4.4.1 Connection and Section APIs

`joinGame(joinCode, role)`

Allows a client to join an existing game session using a valid join code. The role parameter specifies whether the client is joining as a player or spectator.

- **Inputs:**
  - Join code, client role
- **Outputs:**
  - Initial game state snapshot or error response
- **Validation:**
  - Ensures session exists and that player capacity has not been exceeded

Sample Request:

```
{
  "action": "JOIN_GAME",
  "payload": {
    "joinCode": "GREEN-3213",
    "role": "PLAYER",
```

```
    "displayName": "Alice"
  }
}
```

Sample Response:

```
{
  "status": "SUCCESS",
  "data": {
    "clientId": "C-2001",
    "role": "PLAYER",
    "gameState": {
      "gameId": "G-1001",
      "phase": "LOBBY",
      "players": [
        {
          "playerId": "P-1",
          "name": "Alice",
          "resources": {
            "WOOD": 0,
            "BRICK": 0,
            "WHEAT": 0,
            "SHEEP": 0,
            "ORE": 0
          },
          "victoryPoints": 0
```

```
        }
      ],
      "spectators": [],
      "boardState": {
        "hexTiles": [],
        "structures": []
      }
    }
  }
}
```

Error Response (Section does not exist):

```
{
  "status": "ERROR",
  "error": {
    "code": "SESSION_NOT_FOUND",
    "message": "No game session found for join code
GREEN-3123."
  }
}
```

Error Response (Player capacity exceeded):

```
{
  "status": "ERROR",
  "error": {
    "code": "PLAYER_CAPACITY_EXCEEDED",
    "message": "Maximum number of players reached."
  }
}
```

`createGame(config)`

Creates a new game session and generates a unique join code.

- **Inputs:**
  - Game configuration parameters (e.g. player limit)
- **Outputs:**
  - Generated join code and lobby state
- **Validation:**
  - Ensures configuration parameters are within allowed bounds

Sample Request:

```
{
  "action": "CREATE_GAME",
  "payload": {
    "config": {
      "maxPlayers": 4,
      "allowSpectators": true,
      "boardSize": "STANDARD",
      "winConditionPoints": 10
```

```
    }
  }
}
```

Sample Response:

```
{
  "status": "SUCCESS",
  "data": {
    "joinCode": "GREEN-4234",
    "lobbyState": {
      "gameId": "G-1001",
      "maxPlayers": 4,
      "players": [],
      "spectators": [],
      "phase": "LOBBY"
    }
  }
}
```

Error Response:

```
{
  "status": "ERROR",
  "error": {
    "code": "INVALID_CONFIGURATION",
    "message": "maxPlayers must be between 2 and 4.",
    "details": {
      "providedMaxPlayers": 6,
      "allowedRange": "2-4"
    }
  }
}
```

4.4.2 Gameplay Action APIs

`rollDice()`

Requests a dice roll during the active player's turn.

- **Inputs:**
  - None (implicitly bound to requesting client)
- **Outputs:**
  - Dice result and updated resource allocations
- **Validation:**
  - Confirms the requesting client is the active player and the game is in the ROLL phase

Sample Request:

```
{
  "action": "ROLL_DICE"
}
```

Sample Response:

```
{
  "status": "SUCCESS",
  "data": {
    "diceResult": 8,
    "resourcesDistributed": [
      {
        "playerId": "P-1",
        "resourceType": "WOOD",
        "amount": 1
      },
      {
        "playerId": "P-2",
        "resourceType": "BRICK",
        "amount": 1
      }
    ],
    "nextPhase": "BUILD_PHASE"
  }
}
```


Error Response:

```
{
  "status": "ERROR",
  "error": {
    "code": "INVALID_PHASE",
    "message": "Dice can only be rolled during ROLL_PHASE.",
    "details": {
      "currentPhase": "BUILD_PHASE"
    }
  }
}
```

`buildStructure(type, location)`

Requests construction of a structure (e.g., road or settlement) at a specified location.

- **Inputs:**
  - Structure type and board location
- **Outputs:**
  - Updated game state
- **Validation:**
  - Checks resource availability, placement legality, and turn phase

Sample Request:

```
{
  "action": "BUILD_STRUCTURE",
  "payload": {
    "type": "SETTLEMENT",
    "location": {q: 12, r: 4}
  }
```

```
}
```

Sample Response:

```
{
  "status": "SUCCESS",
  "data": {
    "updatedResources": {
      "WOOD": 1,
      "BRICK": 1,
      "WHEAT": 0,
      "SHEEP": 0,
      "ORE": 0
    },
    "updatedStructures": [
      {
        "structureId": "S-3001",
        "ownerId": "P-1",
        "type": "SETTLEMENT",
        "location": {q: 12, r: 4}
      }
    ],
    "updatedVictoryPoints": 2
  }
}
```

Error Response (Illegal Placement):

```
{
  "status": "ERROR",
  "error": {
    "code": "ILLEGAL_PLACEMENT",
    "message": "Settlement cannot be placed at NODE-42-4.",
    "details": {
      "reason": "Adjacent node already occupied."
    }
  }
}
```

Error Response (Insufficient Resources):

```
{
  "status": "ERROR",
  "error": {
    "code": "INSUFFICIENT_RESOURCES",
    "message": "Not enough resources to
build SETTLEMENT."
  }
}
```

`endTurn()`

Signals completion of the active player's turn.

- **Inputs:**
  - None
- **Outputs:**
  - Updated turn index and phase
- **Validation:**
  - Ensures all mandatory actions for the turn have been completed

Sample Request:

```
{
  "action": "END_TURN"
}
```

Sample Response:

```
{
  "status": "SUCCESS",
  "data": {
    "previousPlayerId": "P-1",
    "nextPlayerId": "P-2",
    "newPhase": "ROLL_PHASE"
  }
}
```

Error Response:

```
{
  "status": "ERROR",
  "error": {
    "code": "MANDATORY_ACTION_INCOMPLETE",
    "message": "Player must roll dice before ending turn."
  }
}
```

4.4.3 State Synchronization APIs

`gameStateUpdate(state)`

Server-initiated broadcast that synchronizes the authoritative game state to all connected clients.

- **Broadcast:**
  - Updated game state snapshot
- **Validation:**
  - Ensures all clients render a consistent view of the game

Broadcast:

```
{
  "event": "GAME_STATE_UPDATE",
  "data": {
    "gameId": "G-1001",
    "phase": "BUILD_PHASE",
    "currentTurnPlayerId": "P-2",
    "players": [
      {
        "playerId": "P-1",
        "victoryPoints": 2,
        "resourceCount": 4
      },
      {
        "playerId": "P-2",
        "victoryPoints": 1,
        "resourceCount": 3
      }
    ],
    "boardState": {
      "structures": [],
      "robberPosition": "HEX-5"
```

```
    }
  }
}
```

4.4.4 Error and Validation Responses

`actionRejected(reason)`

Notifies a client that a requested action was rejected

- **Inputs:**
  - Human-readable rejection reason
- **Purpose:**
  - Prevents silent failures and supports client-side feedback

Sample Response (Reason):

```
{
  "event": "ACTION_REJECTED",
  "data": {
    "reason": "You are not the active player.",
    "code": "NOT_ACTIVE_PLAYER"
  }
}
```

Sample Response (Standardized):

```
{
  "status": "ERROR",
  "error": {
    "code": "ACTION_REJECTED",
    "message": "Requested action was rejected by server validation.",
    "details": {
      "phase": "ROLL_PHASE",
      "playerId": "P-3"
    }
  }
}
```

4.4.5 API Design Rationale

The API design enforces a strict separation between client intent and server authority. By limiting clients to declarative requests and centralizing validation on the server, the system prevents illegal game states, simplifies debugging, and ensures deterministic turn progression. Socket-based APIs further enable low-latency updates and seamless spectator synchronization.

5 - Critical Sections

5.1 Risk Analysis

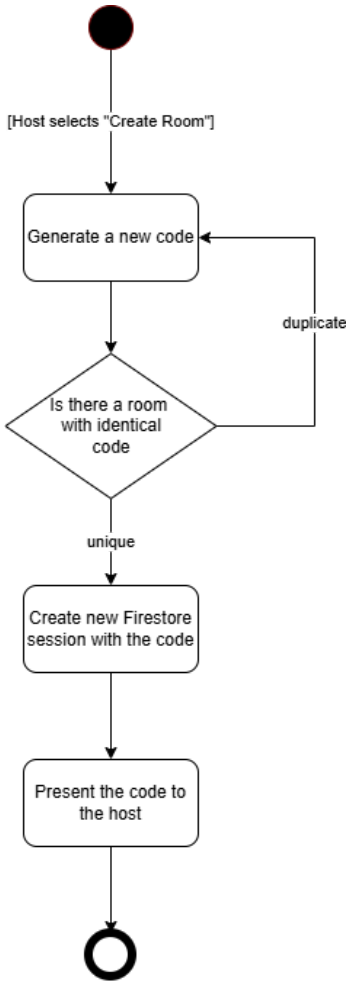
Items	Title	Description	Location	Owner
1	Game Session Creation	Generate unique join code and create Firestore session safely	Host → Server	Developer
2	Room Join Validation	Ensure session exists and is not full before adding player	Join → Waiting Room	Developer
3	Real-time State Synchronization	Ensuring all players receive consistent game state updates via Socket.io	GameBoard	Developer

4	Player Action Validation	Validate all client actions server-side to prevent illegal moves	Server Socket Layer	Developer
5	Disconnect Handling	Maintain game integrity if a player disconnects mid-session	Waiting Room / GameBoard	Developer
6	Game End Finalization	Preventing duplicate result computation and multiple result writes	GameBoard → Result	Developer

5.2 Mitigation Models

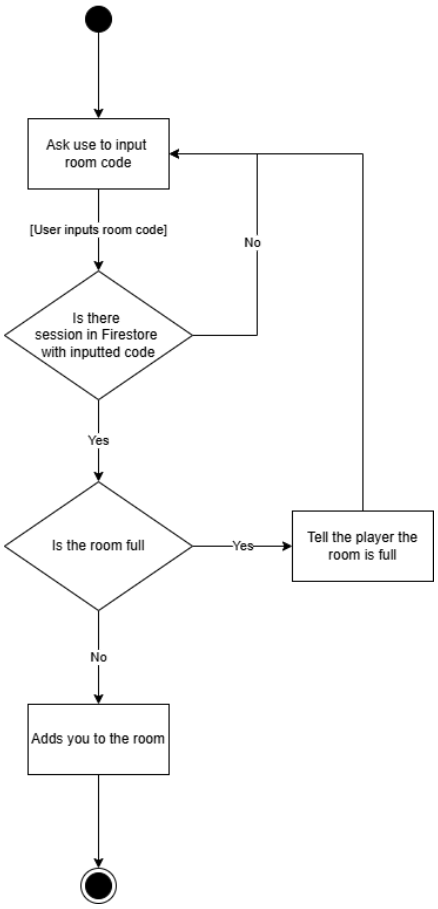
5.2.1 Game Session Creation

This critical section is about making every newly hosted lobby unique and the different game sessions don't interfere with each other in the database. The mitigation algorithm we came up with generates a code at the start of each session, so when a player hosts a new session. The program checks if any other currently running games share the same code as the newly generated one, and if there is, then it generates a new code and repeats the process. When we find a code that is different from those already in session, then it will be presented to the host so that he can share the code to the other players.



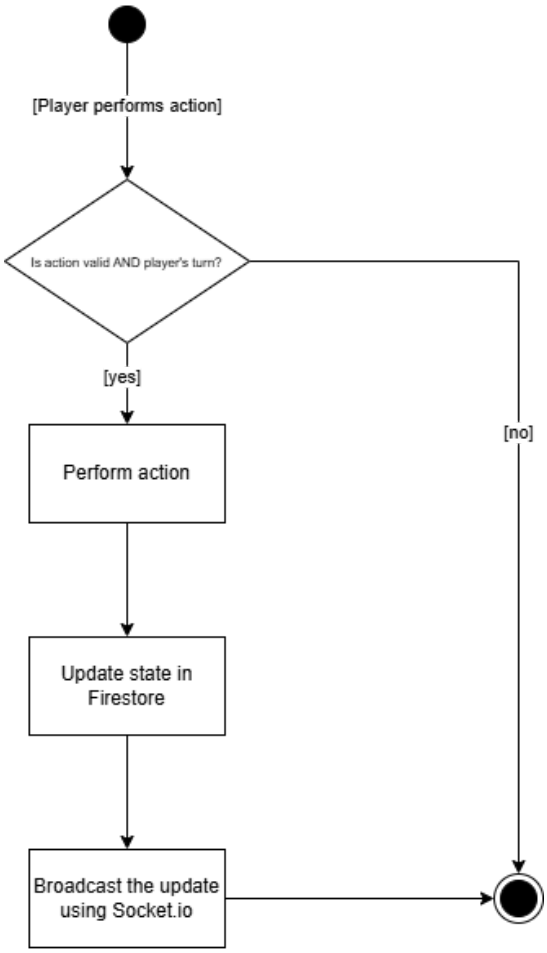
5.2.2 Room Join Validation

This makes sure the player joins the correct room and the room is not full. It first checks if the inputted code has an associated game session with it. Then if it is the case, it makes sure that the room is not already full. If all goes correct, it will add you to the room and update the session database with the newly added player. If a concern is that checking if a game session with said code is deemed to be too long, we can use a sorting algorithm to sort the game sessions by their code and use a search algorithm to make the search proceed faster.



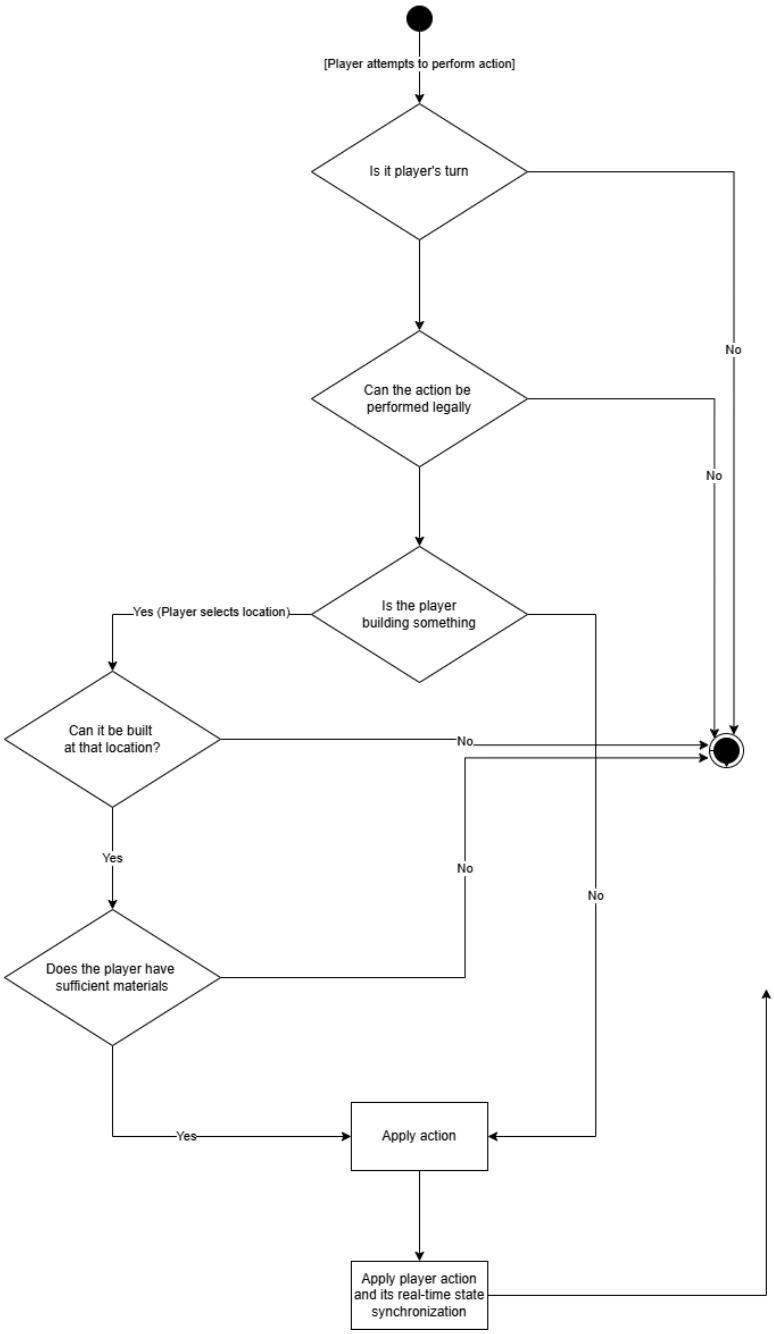
5.2.3 Real-time State Synchronization

This algorithm ensures the shared game state remains consistent. When a player performs an action, it checks if the action is valid using essentially the Player Action Validation algorithm and if it does, it updates the state in the Firestore database. After the database is updated, it will push the update via Stocket.io to all members such as spectators and the players themselves. We treat the game as a single thread and since only the turn player can act, there should be no latency issues.



5.2.4 Player Action Validation

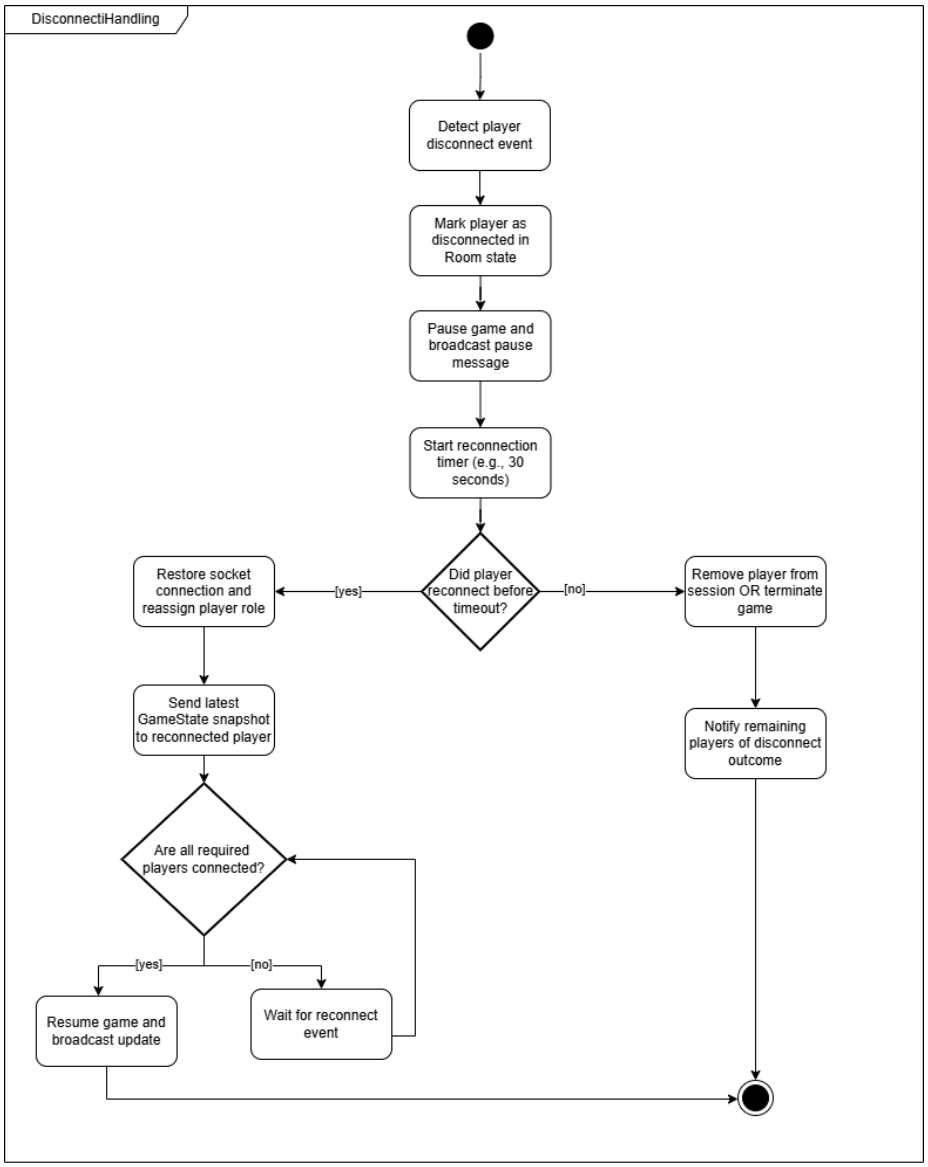
This algorithm ensures that when a player attempts to make an action, it makes sure the action is legal within the context of the game. The mitigation algorithm first makes sure that the player is in fact playing during his turn. In our game, a player can only act during their turn. Then, it makes sure that the click is legal within the current context of the game, such as you cannot roll the dice after already rolling it once. If the player is attempting to build an establishment or a road, it checks if the placement of such is allowed within the rules of the game, then checks if the material requirement is met. After all this, the action can be processed and you can refer to the Real-Time State Synchronization algorithm to make sure the action is processed properly for everyone.



5.2.5 Disconnect Handling

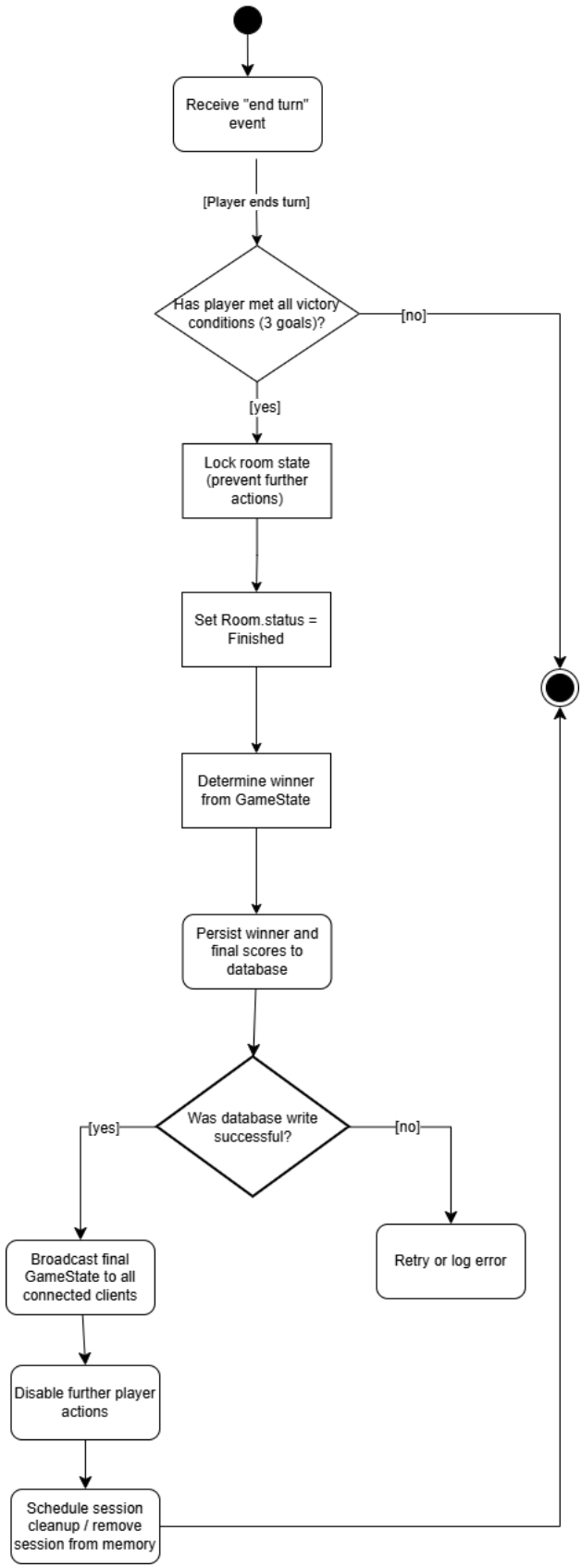
This algorithm ensures that when a player leaves, the game maintains an integral state. When the game detects that a player has left the lobby, it marks the player as disconnected and will pause the game and display a message to all players. Then, it will start a timer for say 30 seconds (TBD). After 30 seconds have passed, if the player has reconnected, the game will attempt to reinstate to the previous game state. Then, it will make sure that all the required players are connected. If not, it will repeat this process for the other disconnected players. If all players are present, then the game will resume. However, if the

disconnected player did not reconnect after the time frame, then the host will be prompted to keep waiting or can choose to either remove the player from the game or terminate the game entirely. Everyone else will then be notified by the change. Then, the game can resume.



5.2.6 Game End Finalization

Our game-end finalization mitigation algorithm ensures the match terminates exactly once and that the final outcome cannot be corrupted by late or concurrent actions. When an End Turn event is received, the server re-evaluates victory conditions using the authoritative GameState. If a win condition is met, the server locks the room to prevent any further player actions, sets the room status as finished, and sets the winner and final scores in the database. If the update is successful, it will display the result to all players and spectators and disable any further actions from all parties. Then, it will schedule a session cleanup in order to remove the session from the memory.


Receive "end turn" event
[Player ends turn]
Has player met all victory conditions (3 goals)?
[no]
[yes]
Lock room state (prevent further actions)
Set Room.status = Finished
Determine winner from GameState
Persist winner and final scores to database
Was database write successful?
[yes]
[no]
Broadcast final GameState to all connected clients
Retry or log error
Disable further player actions
Schedule session cleanup / remove session from memory

6 Appendix

Appendix A - Contribution Table

Student Name	Student ID	Sections Contributed	Notes
Nathan Hu	261147733	Appendix, 4.4	
Helena Clifford	261104576	1	
Barry Lu	261091921	2.2-2.4, 3.1	Shared 3.1 with Emily Zhang
Austin Wang	260349977	4.2-3	4.2 shared with Brandon Lee Felix
Emily Zhang	261177806	2,3	
James Huang	261140630	5	
Brandon Lee Felix	261234731	4.0-2	

Appendix B — References

Google. “Cloud Firestore Documentation.” Firebase Documentation,

https://firebase.google.com/docs/firestore. Accessed 11 Feb. 2026.

Appendix C — Digital Source Locations

Artifact	Location (URL/Path)	Owner
GitHub Repo	https://github.com/Nhucanada/HexaHaven	Nathan Hu
StoryBoard	https://drive.google.com/file/d/1hjmNJaTiQZkyJemun8Tbq831NIwszCtb/view?usp=sharing	Barry Lu
Deployment Diagram	https://app.diagrams.net/#G1KSEKWdMdTqMy3H7uaB8FmszHOPqIbRvd#%7B"pageId"%3A"f1wxPazErdoyT4Uih1ah"%7D	Emily Zhang
UML Diagram	https://app.diagrams.net/#G1nt35KJS0pkE8ikO5ktetR8G9MSw7YPSl#%7B%22pageId%22%3A%22FxbZokW7ckKsJ4nu6k3_%22%7D	Emily Zhang
Database Diagram	https://drive.google.com/file/d/1MWk1BQpXLmsHIZbHIZ-9YOZO-LhmjivH/view?usp=sharing	Austin Wang
Critical Section: Game Session Creation	https://drive.google.com/file/d/1L2Chyi5alg3vLn_0qMV-Q93Wf1UStw1g/view?usp=sharing	James Huang
Critical Section: Room Join Validation	https://drive.google.com/file/d/1XtQ2y8Z4-mwXcXAzr0UIR_28xDNplaG7/view?usp=sharing	James Huang
Critical Section: Real-time State Synchronization	https://drive.google.com/file/d/1nQbNCQJclxpTc5VmdpnsOykHKqubxVys/view?usp=sharing	James Huang

Critical Section: Player Action Validation	https://drive.google.com/file/d/1blvsnUZVW96cgpurGqPBPnfo9Q_AsKlN/view?usp=sharing	James Huang
Critical Section: Disconnect Handling	https://drive.google.com/file/d/1QgrjRHkxecWr7Xy6ihsXYwcuaJxt1Arr/view?usp=sharing	James Huang
Critical Section: Game End Finalization	https://drive.google.com/file/d/1TMyaZpmekwCEXRJejwZoGlzvmsRs5gk4/view?usp=sharing	James Huang

Appendix D — Index

1. **Action Object**
   Structured record stored within a turn document representing a single validated player action (e.g., DICE_ROLL, BUILD_STRUCTURE, GOAL_COMPLETED).

2. **Active Player**
   The player whose turn is currently being processed according to the authoritative GameState.

3. **ActiveGames Subcollection**
   Firestore subcollection under /users/{userId} used to track a user's active or recent sessions for quick dashboard lookup.

4. **Authoritative Server**
   Server model in which all game rules, state transitions, and validations are handled exclusively on the server to prevent inconsistencies or cheating.

5. **Axial Coordinates**
   Coordinate system (q, r) used to represent positions on the hex-based board.

6. **Backward Compatibility**
   Schema evolution principle ensuring new database fields do not break existing persisted game sessions.

7. **Broadcast**
   Server-initiated Socket.io event that synchronizes the authoritative game state to all connected clients.

8. **Client / Server Architecture**
   System design where the client handles rendering and input collection, while the server manages game logic, state, validation, and synchronization.

9. **Composite Index**
   Firestore index combining multiple fields to support specific query patterns (e.g., ordering turns by turnNumber).

10. **Data Isolation**
    Separation of staging and production Firestore data to prevent test sessions from affecting live sessions.

11. **Desynchronization**
    State mismatch between clients or between client and server, prevented by server-authoritative validation and broadcast synchronization.

12. **Dice Roll**
    The core gameplay mechanic used to trigger resource production during a player's turn.

13. **Distributed System**
    Architecture where multiple clients interact with a centralized authoritative server over a network.

14. **Edge**
    Board element representing a connection between two vertices; roads are built on edges.

15. **EdgeLocation**
    Structured object describing the exact board location of a road using hex coordinates and direction index.

16. **Event-Driven Architecture**
    Design pattern where system behavior is triggered by events (e.g., JOIN_GAME, ROLL_DICE) transmitted via WebSocket communication.

17. **Firebase Admin SDK**
    Server-side SDK used to securely interact with Firestore and enforce authoritative validation.

18. **Firebase Emulator Suite**
    Local testing environment used to simulate Firestore behavior during development and staging.

19. **Firestore Collections**
    Primary document groupings used for persistent storage (e.g., /games, /users).

20. **Firestore Security Rules**
    Access control policies defining valid read/write permissions and integrity constraints.

21. **Firestore Transactions**
    Atomic operations ensuring consistent state updates during concurrent multiplayer interactions.

22. **Game Board**
    The main gameplay screen displaying the hex-based map, player structures, resources, and turn information.

23. **GameConfig**
    Nested configuration object within a game document defining player count, win rules, map size, and starting resources.

24. **Game End Finalization**
    Critical section ensuring victory conditions are evaluated once, persisted atomically, and broadcast safely without duplication.

25. **Game Session Lifecycle**
    Full progression of a game from creation, lobby, gameplay, completion, and archival or expiration.

26. **Game State**
    The complete representation of the current game session, including players, board layout, resources, turn order, and goals.

27. **Goal**
    Nested object defining player objectives, completion status, target values, and progress tracking.

28. **Hex Map / Hex-Based Board**
    A grid of hexagonal tiles used as the main game board layout, where each tile may produce resources.

29. **HTML Canvas**
    Client-side rendering surface used to draw the hex-based game board dynamically in the browser.

30. **Indexing (Firestore)**
    Definition of Firestore indexes required to support efficient querying (e.g., roomCode lookup, ordering turns by turnNumber).

31. **Interim Demonstration**
    Scheduled project checkpoints where partial functionality is demonstrated to verify progress.

32. **Join Code**
    A short identifier used by players or spectators to join an existing game session.

33. **JSON Structure**
    Structured data representation format used internally within Firestore documents and API payloads.

34. **Lobby / Waiting Room**
    Pre-game screen where players gather, view available slots, and wait for the host to start the game.

35. **MVC Pattern**
    Design pattern separating game state (Model), UI rendering (View), and user interaction logic (Controller).

36. **Observer Pattern**
    Design pattern allowing clients to subscribe to server updates and automatically receive state changes.

37. **Persistence Layer**
    Server component responsible for interacting with Firestore to store and retrieve authoritative game data.

38. **Phase Validation**
    Server-side enforcement ensuring actions occur only during valid phases (ROLL, COLLECT, ACTION, END).

39. **PresenceInfo**
    Nested schema storing player connection state, including isConnected and lastSeenAt.

40. **Real-Time Synchronization**
    Mechanism ensuring all clients maintain consistent state through authoritative validation and broadcast updates.

41. **Resource**
    In-game items (e.g., CRYSTAL, STONE, BLOOM, EMBER, GOLD) collected by players and used to build structures or complete goals.

42. **ResourceBundle**
    Structured object representing quantities of all resource types owned by a player or required for construction.

43. **Road**
    Structure built on board edges used for expansion and longest road evaluation.

44. **RoadPath**
    Structured object describing connected roads and path length for goal validation.

45. **Schema Evolution**
    Controlled modification of Firestore document structures while preserving compatibility with existing sessions.

46. **Server Validation**
    Process by which the authoritative server checks legality of all client-submitted actions before applying state changes.

47. **Session Expiration Policy**
    Maintenance rule for cleaning up stale or inactive sessions from Firestore.

48. **Singleton Pattern**
    Design pattern ensuring one authoritative GameServer instance manages sessions.

49. **Snapshot Listener**
    Firestore real-time subscription mechanism used to detect document changes.

50. **Soft Deletion**
    Data management technique using a boolean flag (isDeleted) instead of permanently removing documents.

51. **Socket.io**
    Real-time communication library used to send player actions to the server and broadcast game state updates to all clients.

52. **Spectator Mode**
    Read-only mode allowing users to watch an active game without interacting with or affecting gameplay.

53. **State Divergence**
    Condition where clients display inconsistent game states; mitigated through authoritative

broadcast enforcement.

54. **Structure**
    Generic term for buildable board entities such as roads and settlements.

55. **Subcollection**
    Firestore nested collection under a document (e.g., /games/{gameId}/players/{playerId}).

56. **Turn Log**
    Subcollection (/games/{gameId}/turns) storing sequential records of each turn and associated actions.

57. **Turn Phase**
    Current sub-state of a player's turn (ROLL, COLLECT, ACTION, END).

58. **Turn State Machine**
    Logical model controlling legal transitions between phases within a turn.

59. **Turn-Based Gameplay**
    Game structure where players act sequentially according to a defined turn order.

60. **Validation Response**
    Standardized error or success response returned by the server after processing an action request.

61. **Vertex**
    Board intersection point where settlements may be placed.

62. **VertexLocation**
    Structured object identifying the exact location of a settlement on the board.

63. **Victory Points**
    Numerical scoring system used to determine win conditions.

64. **Web Browser**
    Execution environment on the client device responsible for loading UI files, rendering the game, and running client-side logic.

65. **WebSocket**
    Persistent network communication protocol enabling low-latency multiplayer interaction.